



Bicol University
POLANGUI CAMPUS
Centro Occidental Polangui, Albay



Performance Report (Week 15) in IT 112: Web Systems and Technologies
2nd Semester A.Y 2025 - 2026

NEXUSWrites
“Your Tech, Your Story.”

Group Name:
Powerpuffgals

Group Members:
Kris Ann B. Apoon
April Grace R. Dacanay
Ivy Angel R. Hidalgo
Princes Mae L. Mateo
Milan R. Rellora

Course & Block:
BSIT-2A



Bicol University
POLANGUI CAMPUS
Centro Occidental Polangui, Albay



Introduction

In this phase of NEXUSWrites, the website becomes more engaging as it starts displaying real blog content retrieved from the backend. Instead of static text, blog posts and related data are now loaded dynamically and presented on the webpage. This allows users to view actual content, making the platform feel more active and closer to a real blogging site.

This stage is important because it improves both the functionality and experience of the website. By using JavaScript and DOM manipulation, blog posts can be displayed in an organized and visually appealing way. Features such as structured layouts and optional search tools help users easily navigate through different posts, making the platform more interactive and user-friendly.

Objectives

By the end of this phase, we were able to:

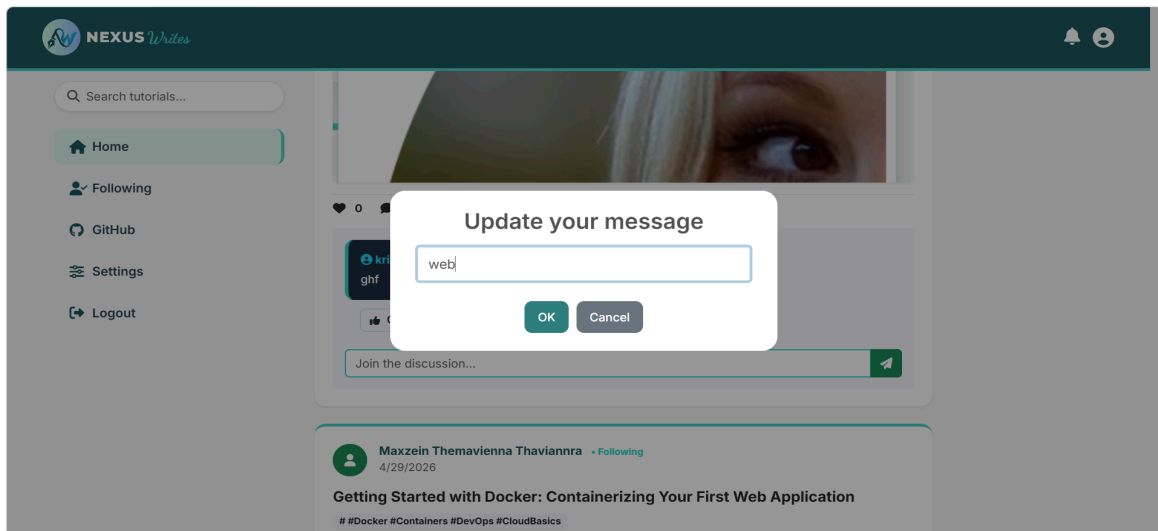
- Implement an interactive and intuitive user interface for editing existing comments, featuring a pre-populated modal for modification, user confirmation options, and clear post-edit feedback.
- Develop a secure and user-friendly interface for deleting posts, incorporating a confirmation modal with irreversible action warnings to prevent accidental data loss and ensure user intent.
- Create robust client-side JavaScript functions for handling comment editing and post deletion, including mechanisms for data retrieval, input validation (e.g., preventing empty submissions), and responsive user feedback through modal interactions.
- Establish authenticated backend API routes and controller logic for post operations, ensuring secure creation, updating, and deletion of content, and verifying user authorization (e.g., author ownership) for modification and deletion actions, alongside enabling image uploads for relevant operations.

I. Working Edit Modal or Page

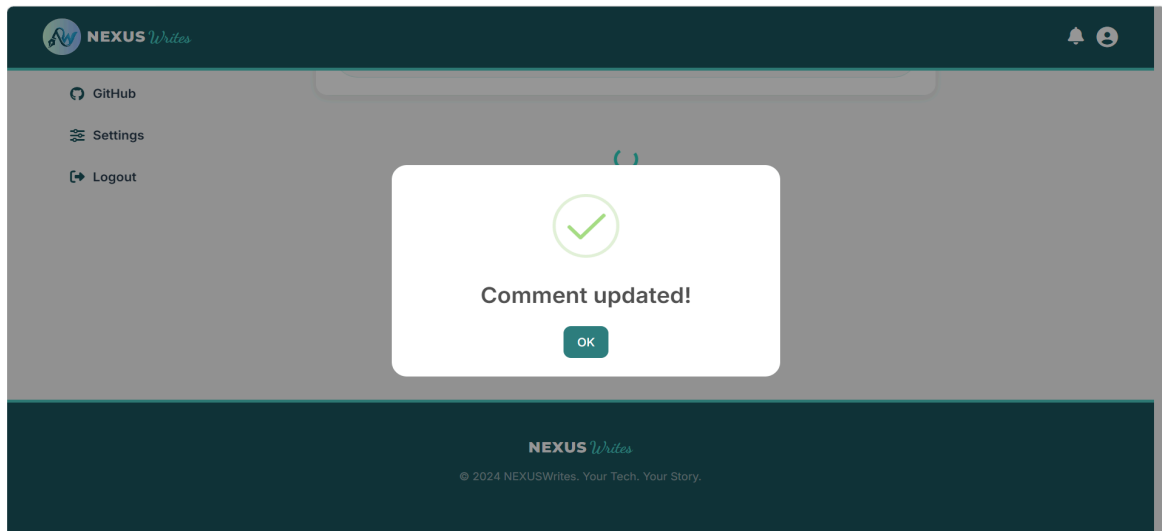
This part shows the user interface process for editing comments: first, a pop-up box appears with the original text, allowing users to modify it and choose to confirm or cancel. After a successful edit, another pop-up with a checkmark and confirmation message appears, letting users know the update is complete.



Bicol University
POLANGUI CAMPUS
Centro Occidental Polangui, Albay



This photo depicts the process of updating a message, which is handled by a PUT request. The relevant route is `router.put('/:id', protect, upload.single('image'), postController.updatePost);`, which triggers the `exports.updatePost` function.



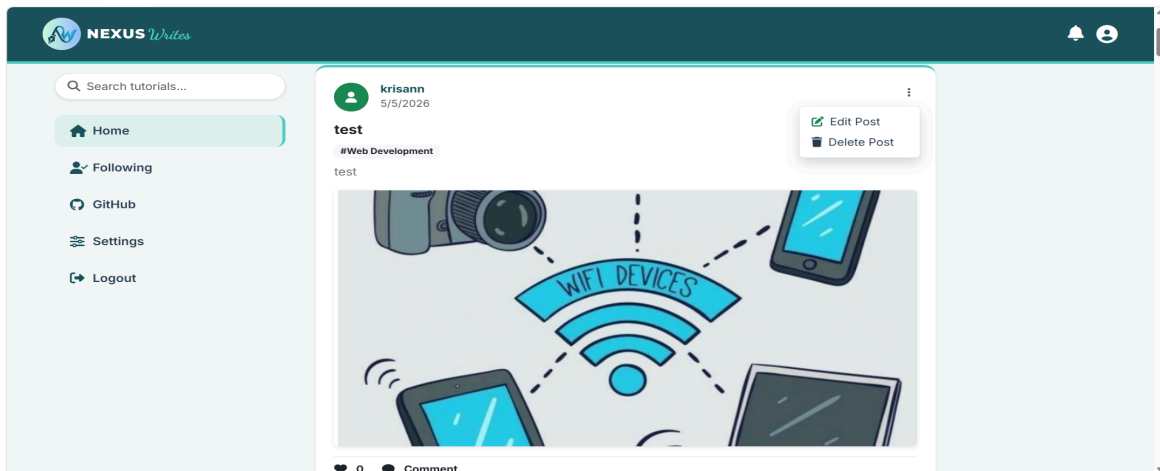


Bicol University
POLANGUI CAMPUS
Centro Occidental Polangui, Albay

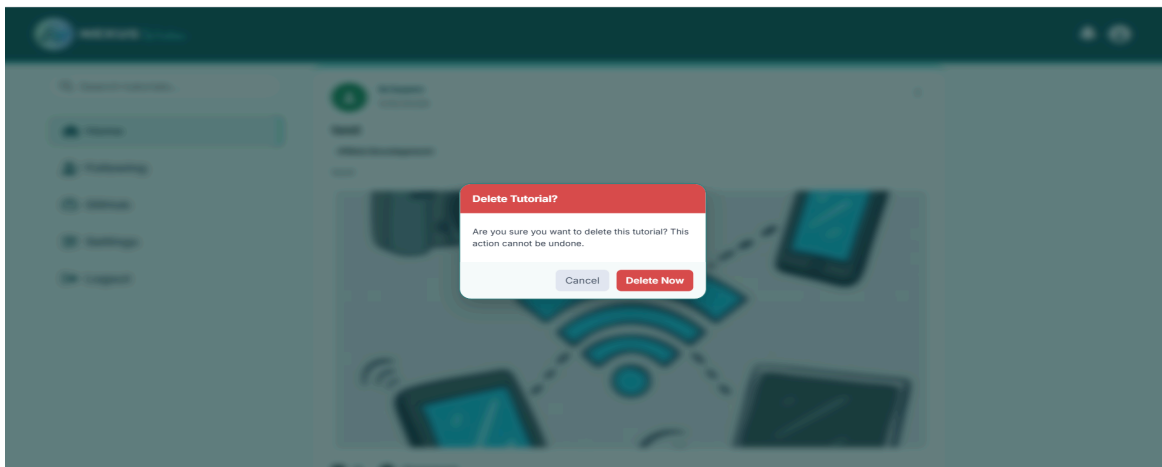


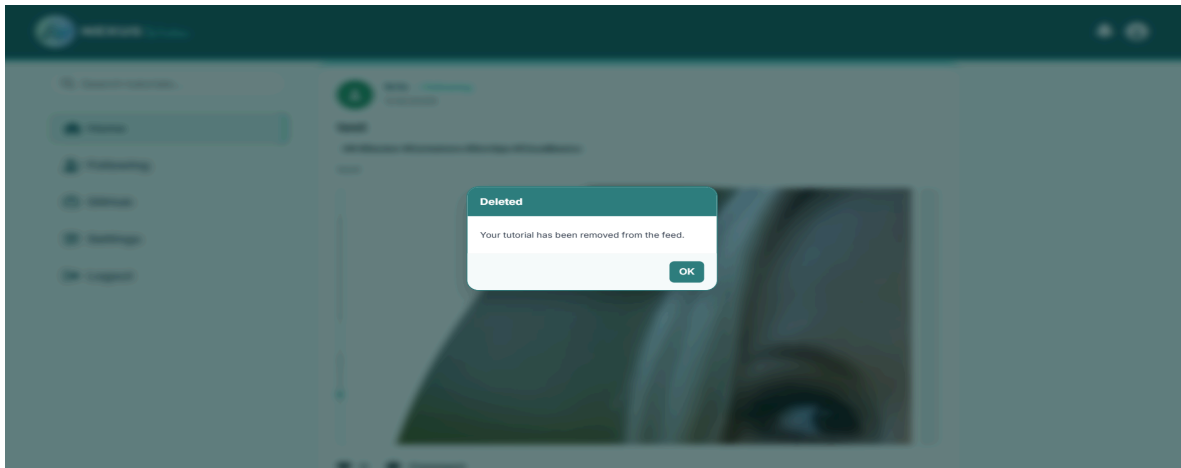
II. Deletion Confirmation and Data Updates

This section demonstrates the interface flow for deleting posts. When selecting "Delete Post" from the options menu, a warning pop-up appears stating the action cannot be undone, giving users the choice to cancel or proceed with deletion to prevent accidental removal of content.



This photo shows a confirmation for deleting a tutorial, which is managed by a DELETE request. The corresponding route is `router.delete('/:id', protect, postController.deletePost);`, executing the `exports.deletePost` function.





III. JavaScript Snippets for Edit and Delete

These are client-side codes handling edit and delete actions. The edit function gets the original comment text, opens an input modal with validation to ensure no empty submissions, and stops if no changes are made. The delete function triggers a warning modal to inform users of permanent removal and related effects before proceeding.

```

320 // --- 1. EDIT COMMENT ---
321 async function editComment(postId, commentId) {
322   const textElement = document.getElementById("text-${commentId}");
323   const oldText = textElement ? textElement.innerText : "";
324
325   // Modern input modal
326   const { value: newText } = await Swal.fire({
327     title: 'Update your message',
328     input: 'text',
329     inputValue: oldText,
330     showCancelButton: true,
331     confirmButtonColor: '#207D7D',
332     cancelButtonColor: '#6c757d',
333     inputValidator: (value) => {
334       if (!value) return 'Message cannot be empty!';
335     }
336   });
337
338   if (!newText || newText === oldText) return;

```

```

367 // --- 2. DELETE COMMENT ---
368 async function deleteComment(postId, commentId) {
369   const result = await Swal.fire({
370     title: 'Delete Comment?',
371     text: "This will also remove all replies!",
372     icon: 'warning',
373     showCancelButton: true,
374     confirmButtonColor: '#d33', // Danger red
375     cancelButtonColor: '#207D7D', // Nexus Teal
376     confirmButtonText: 'Yes, delete it!'
377   });
378
379   if (!result.isConfirmed) return;
380
381   const user = JSON.parse(localStorage.getItem('nexusUser'));
382   if (!user || !user.token) return Swal.fire("Error", "Please log in to perform this action.", "error");
383

```



Bicol University
POLANGUI CAMPUS
Centro Occidental Polangui, Albay



IV. Backend Routes and Controller Snippets

This part defines API routes for post operations, where viewing a post is public while creating, updating, or deleting requires authentication, and image upload is enabled for create and update actions. It also shows the start of the update function, which checks if the post exists and verifies that the logged-in user is the original author before processing data like tags.

```
26 // 4. General Parameter Routes
27 router.get('/:id', postController.getPostById);
28 router.post('/', protect, upload.single('image'), postController.createPost);
29 router.put('/:id', protect, upload.single('image'), postController.updatePost);
30 router.delete('/:id', protect, postController.deletePost);
31
32 module.exports = router;
```

```
188 // 7. EDIT POST
189 exports.updatePost = async (req, res) => {
190   try {
191     const post = await Post.findById(req.params.id);
192     if (!post) return res.status(404).json({ message: "Post not found" });
193
194     // --- STEP 1: FIX THE 401 UNAUTHORIZED ---
195     const authorId = post.author._id ? post.author._id.toString() : post.author.toString();
196     const currentUserId = req.user._id.toString();
197
198     if (authorId !== currentUserId) {
199       return res.status(401).json({ message: "Unauthorized: You don't own this post." });
200     }
201
202     // --- STEP 2: FIX THE TAGS ---
203     if (req.body['tags[]']) {
204       post.tags = Array.isArray(req.body['tags[]']) ? req.body['tags[]'] : [req.body['tags[]']];
205     } else if (req.body.tags) {
206       // Added a fallback in case your edit modal sends 'tags' instead of 'tags[]'
207       post.tags = Array.isArray(req.body.tags) ? req.body.tags : [req.body.tags];
208     }
209   }
```



Bicol University
POLANGUI CAMPUS
Centro Occidental Polangui, Albay



This section continues the update function, updating only the fields sent by the user and keeping existing values if nothing new is provided, and updating the image path if a new file is uploaded. After saving changes, it refetches the post with full author details and sends it back, while any errors are caught and returned with a 500 status code.

```
210 // --- STEP 3: UPDATE OTHER FIELDS ---
211 post.title = req.body.title || post.title;
212 post.content = req.body.content || post.content;
213 post.category = req.body.category || post.category;
214
215 if (req.file) {
216   post.image = req.file.path; // Cloudinary upload
217 }
218
219 await post.save();
220
221 // --- STEP 4: RE-POPULATE ---
222 const updatedPost = await Post.findById(post._id).populate('author', 'name avatar bio');
223
224 res.json({ message: "Post updated! 🚀", post: updatedPost });
225
226 } catch (err) {
227   res.status(500).json({ error: err.message });
228 }
229 };
230
```

This is the delete function, which first checks if the post exists and confirms the user owns it to prevent unauthorized deletion. If all checks pass, the post is removed from the database and a success message is sent, with server errors handled in the same way as other functions.

```
231 // 8. DELETE POST
232 exports.deletePost = async (req, res) => {
233   try {
234     const post = await Post.findById(req.params.id);
235
236     if (!post) {
237       return res.status(404).json({ message: "Post not found" });
238     }
239
240     const authorId = post.author._id ? post.author._id.toString() : post.author.toString();
241     const currentUserId = req.user._id.toString();
242
243     if (authorId !== currentUserId) {
244       return res.status(401).json({ message: "Unauthorized: You can only delete your own tutorials." });
245     }
246
247     await Post.findByIdAndDelete(req.params.id);
248     res.json({ message: "Post deleted. 🗑️" });
249   } catch (err) {
250     res.status(500).json({ error: err.message });
251   }
252 };
```



Bicol University
POLANGUI CAMPUS
Centro Occidental Polangui, Albay



V. GitHub Commit Logs

Commit Message	User	Commit Hash
Merge branch 'main' of https://github.com/itzmmr/final-project-Powerpuffgals-BSIT2a-	itzmmr committed 2 days ago	0bace16
Added new frontend screenshot for lab11	itzmmr committed 2 days ago	4ca96b0
style: Update layout and CSS for CRUD modals and feedback messages	ivyangel-hidalgo committed 2 days ago	5217dfc
feat: Add delete confirmation prompts and notification handling	ivyangel-hidalgo committed 2 days ago	bb6f6b0
feat: Implement prefilled forms and PUT request handling for updates	ivyangel-hidalgo committed 2 days ago	1383ab4
feat: Add edit/delete buttons and modal structures to core pages	ivyangel-hidalgo committed 2 days ago	2375d48
feat: Implement PUT and DELETE endpoints for users, posts, and comments	ivyangel-hidalgo committed 2 days ago	cbf17b8

VI. Member Reflections

Ivy Angel Hidalgo (Project Manager & Backend Developer)

As the backend developer, I was responsible for handling how the system processes and updates data. Implementing the PUT and DELETE functions allowed me to better understand how the system works behind the interface. This experience helped me improve my logical thinking and problem-solving skills.

Kris Ann Apoon (Frontend Developer)

As the frontend developer, I focused on making the interface clear and easy to use. I worked on the add, edit, and delete features, and ensured that the buttons and forms function properly. This role helped me become more attentive to details and how users interact with the system.

April Grace Dacanay (Documentation Officer)

As the documentation officer, I prepared the visuals and notes for our lab report. I made sure that our process was clearly explained and easy to follow. This role helped me improve how I present information in a structured and understandable way.



Bicol University
POLANGUI CAMPUS
Centro Occidental Polangui, Albay



Princes Mae Mateo (Database Manager)

As the database manager, I made sure that all data updates and deletions were correct. This role required me to be careful and consistent in checking the database. It helped me develop a stronger sense of responsibility when handling important information.

Milan Rellora (GitHub Manager)

As the GitHub manager, I handled merging our group's work and keeping our files updated. This role required clear coordination with my groupmates to avoid errors. It helped me improve my organizational skills and understand the importance of proper version control.